SKILL 3

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMAND 100

// Node for command history
typedef struct Node {
char *command;
struct Node *next;
struct Node *prev;
} Node;

Node *head = NULL;
Node *tail = NULL;

// Add command to history
void add_history(const char *cmd) {
Node *newNode = malloc(sizeof(Node));

if (newNode == NULL) {
printf("Memory allocation failed\n");
return;
}
```

```
newNode->command = malloc(strlen(cmd) + 1);

if (newNode->command == NULL) {
free(newNode);
printf("Memory allocation failed\n");
return;
}

strcpy(newNode->command, cmd);

newNode->next = NULL;
newNode->prev = tail;

if (tail != NULL)
tail->next = newNode;
else
head = newNode;

tail = newNode;
}

// Display history
void display_history() {
```

SKILL 3

```c
// Display history
void display_history() {
Node *temp = head;
int count = 1;

while (temp != NULL) {
printf("%d %s\n", count++, temp->command);
temp = temp->next;
}
}

// Free all memory
void free_history() {
Node *temp = head;

while (temp != NULL) {
Node *next = temp->next;

free(temp->command);
free(temp);

temp = next;
}
```

```c
head = NULL;
tail = NULL;
}

int main() {
char input[MAX_COMMAND];

printf("Mini Shell History\n");
printf("Type 'history' to display commands.\n");
printf("Type 'exit' to quit.\n\n");

while (1) {
printf("shell> ");

if (fgets(input, sizeof(input), stdin) == NULL)
break;

// Remove newline
input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
break;
```

```c
if (strcmp(input, "history") == 0) {
display_history();
continue;
}

if (strlen(input) > 0) {
add_history(input);
}
}

free_history();

return 0;
}
```

SKILL 3

```
sai_pavani@Sai-Pavani:~/OS_LAB$ gcc skill3.c -o skill3
sai_pavani@Sai-Pavani:~/OS_LAB$ ./skill3
Mini Shell History
Type 'history' to display commands.
Type 'exit' to quit.

shell> history
shell> exit
sai_pavani@Sai-Pavani:~/OS_LAB$
```